Problem Overview

You are required to design an application model for a pool table game. The pool game is played on a rectangular table with different colour of balls on it. This pool table has 6 pockets into which the balls can fall. All the balls are randomly placed on the table at the beginning of the game. Among all balls, there is a ball being the white cue ball. A player can only hit the white cue ball with the cue stick. If the balls approach the side of the table, they will bounce back. Balls also bounce off other balls. This is a single player game and is won when all the balls, other than the cue ball, are in the pockets. Game is lost when the white cue ball falls into any of the pockets. The score is calculated when a ball falls into a pocket. Duration of the game is clocked until all balls are in the pocket.

Q: What is a pool table game?

A: Some real-world examples could be found here Links to an external site. and here Links to an external site..

Assignment 3 Requirement

In assignment 3, you are going to review and extend an existing implementation of 'Pool Game' (that is not your own) to add features, leveraging your knowledge of OOP, design principles and design patterns. Please find the detailed tasks below:

Implementation Task

What we provide to you?

- Codebase: the codebase you are going to review and extend is provided **here** Download here.

Please note that, your goal is to **extend** and **maintain** this implementation. What this means is to add features to the existing implementation by using OO design principles and appropriate design patterns you have learnt throughout this UoS, **without** breaking the implementation (it runs - rule #1 is don't break working code) or using unnecessary 'hacks'.

You are **not** required to correct the existing design of the implementation - you **must retain** the existing design wherever changes are not required and will be penalised should this be changed without cause (for example, replacing the given implementation with your own assignment 2 code). The idea here is that you work with the existing design rather than against it, and minimise required changes to the existing structure (reasonable, limited scope refactoring to support extensions is encouraged).

What we expect from you?

Your Pool Game is now expected to support the following features in the code:

- Pockets and More Coloured Balls
  - The pool table now has pockets whose positions and radius are configurable and specified in the sample JSON configuration files.
  - At this stage, we consider more configurable coloured balls including
    - red, blue, black, yellow, purple, orange, green and brown.
    - After falling into the pocket, an orange ball and a yellow ball behave the same as a red ball (i.e., disappear immediately),
    - a green ball and a purple ball behave the same as a blue ball (i.e., back to its initial position after the first falling whereas disappear after the second time falling into the pocket),
    - whereas a black ball and a brown ball will be back to its initial position after the first two falling, while disappearing after the third time falling into the pocket.
  - There is also now a visible player-controlled cue stick which can hit the cue ball with variable velocity.

state pattern for active/disappear?

- Difficulty Level
  - There are now three difficulty levels in your game including easy, normal and hard, which correspond to configuration files config_easy.json Download config_easy.json, config_normal.json Download config_normal.json and config_hard.json Download config_hard.json, respectively.
  - The player can choose a level either by clicking on a button or by selecting from a menu or by clicking on a keyboard key (i.e., you ONLY need to implement this feature through one of these three ways).
  - You can set the easy level as the default level displayed to the player OR you can ask the player to choose a level before the start of a game.
  - **Attention Please: you are free to change the values in the sample JSON files whereas you are not allowed to change the structure of the JSON files (i.e., no added, no deleted).**

- Time and Score
  - Duration of the game is clocked until all balls are in the pocket. The game must display on the screen a continually updating time (initially at 0:00).
  - The score is calculated when a ball falls into a pocket. The game must display on the screen a continually updating score (initially at 0) during the level.

observer?

- The ball and its corresponding score after each falling into the pocket can be found in the table:

Colour	red	yellow	green	brown	blue	purple	black	orange
Score	1	2	3	4	5	6	7	8

- Undo and Cheat
  - The player can reset the game to an earlier state (including score, time, ball positions) so that a shot can be undo.
  - This undo functionality can be triggered by button, menu or keyboard action (i.e., you ONLY need to implement this feature through one of these three ways).
  - This must be a single state that is not written to disk, and the state reaching by the subsequent undo function overwrites the existing saved state.
  - The player can do a cheating operation to remove all same coloured ball immediately.
    - This functionality can be triggered by button, menu or keyboard action (i.e., you ONLY need to implement this feature through one of these three ways).
    - Take keyboard action as an example only: pressing the key 'b' on the keyboard will immediately remove all blue balls and add corresponding scores.
    - Attention Please: the balls will be removed immediately which is different from falling into a pocket.

memento

Left to do:
- cue stick
- timer fix

Report Task

Your report in this assignment must concisely cover the followings:

1. Code review on the existing codebase provided to you, which includes
   - discussion on the use of OOP design principles (**be specific** to the given code, a UML snippet needs to be provided)
   - discussion on the use of design patterns (**be specific** to the given code, a UML snippet needs to be provided)
   - discussion on the documentation (e.g., readme file, comments, etc.)
   - discussion on how easy or difficult the given codebase and the above points made it to achieve your required functionality in this assignment and the reason
2. A discussion on your feature extension including
   - Describe the actual extension you have made in your code

    - Highlight your application of OO design principles in your extensions and explain what they motivated you to do and why (**be specific** to your code, a UML snippet needs to be provided)
    - Document at least three design patterns you have used in this assignment and rationalise their usage in terms of SOLID and GRASP (**be specific** to your code, not the pattern in general, a UML snippet needs to be provided).
        - You **must use the GoF design patterns** that we have learnt in this unit in your implementation.
        - Attention Please: you are allowed to reuse a design pattern that you have used in A2 however you are not allowed to document a repeat use case of a design pattern as A2, for example, you are not allowed to discuss applying builder design pattern to create balls, the same for factory method and strategy.
    - Reflect on your extension design, highlighting any outstanding issues or improvements or discussing your impact on the extensibility of the code

3. A UML class diagram of the after-extension version of your codebase, highlighting the design patterns you have used **for your extension** and identifying the participants in each design pattern you have used.

4. Any acknowledgement/reference required.

Submission Details

You are required to submit all assessment items by the due date listed on Canvas.

- Report: Submit your UML class diagram and your report as a **SINGLE** pdf document.
    - If your diagram is unreadable at 200% magnification (maximum zoom with the turnitin tool) then consider how to improve it
    - You must include the entire UML diagram, and you must also include enlargements of specific parts for reference in the discussion
- Code: Submitted separately
    - 'gradle run' will start the game.
    - A readme file named "A3_readme.txt" should cover any point you would like your marker to know
        - how to run your code (e.g., any quirks to run your application).
        - List features (i.e., Pockets and More Coloured Balls, Difficulty Level, Time and Score, and Undo and Cheat) you **have implemented** in your extension.
        - List **the names of the design patterns you have used** in your extension and provide the **corresponding class and file names** regarding these patterns.

        - **Attention Please: Code that is not listed here will not be assessed as part of the design pattern.**
    - Describe how to select difficulty level, how to undo and how to cheat in your code.
    - Any other info you would like your marker to know regarding your implementation.